

SISTEMA SAO BAR 2026

DOCUMENTO MAESTRO DE ESPECIFICACIONES

SAO BAR v1.9

Agosto 2026 (08/26)

Desarrollado por Lucas Luccaroni

SISTEMA SAO BAR 2026
Documento Maestro de Especificaciones

DOCUMENTO MAESTRO DE ESPECIFICACIONES V1.9: SISTEMA SAO BAR 2026

1. RESUMEN EJECUTIVO Y ALCANCE TÉCNICO

El sistema SAO Bar 2026 se define como una infraestructura web multi-terminal de alto rendimiento diseñada para la gestión operativa y administrativa de jornadas nocturnas. Su propósito central es garantizar la integridad transaccional y el control de inventario en un entorno de alta demanda, eliminando las discrepancias de los registros manuales.

La versión **v1.9** consolida todas las funcionalidades de control de bar y suma el **Módulo Bot Asistente de IA (RAG + Tool Use + Capa de Abstracción Multi-Proveedor)**, permitiendo a la administración interactuar mediante lenguaje natural con el conocimiento procedimental del bar y con el estado de inventario y caja en tiempo real.

Concepto de Jornada Nocturna No-Calendario

Dada la naturaleza operativa del establecimiento, el sistema ignora la linealidad de la fecha calendario del servidor. La actividad que inicia un viernes por la noche y concluye la madrugada del sábado se agrupa bajo un identificador único de "Jornada". Esto permite consolidar métricas financieras y de stock sin la fragmentación que causaría un cambio de fecha a medianoche.

Entorno Operativo y Concurrencia

El sistema está dimensionado para la operación simultánea de 4 terminales activas bajo un modelo de datos centralizado:

- **3 Terminales de Comandas:** Dispositivos móviles o laptops destinados a la toma de pedidos en salón y barra por parte de los mozos.
- **1 Terminal de Administración:** Estación central (macOS/Windows) para el monitoreo de caja, auditoría física, cierre de jornada y asistente de IA.

Stack Tecnológico

- **Framework:** Next.js 16 (App Router) para una gestión de rutas y estados optimizada.
- **Base de Datos:** Supabase (PostgreSQL) para la persistencia relacional con integridad ACID y extensión **vector** (pgvector).
- **Motor de Inteligencia Artificial (v1.9):**
 - **Embeddings:** OpenRouter / OpenAI (**openai/text-embedding-3-small** a 768 dimensiones) / Hugging Face.
 - **LLM:** OpenRouter (**meta-llama/llama-3.3-70b-instruct**) como motor principal y Groq Cloud Native (**llama-3.3-70b-versatile**) como respaldo.
 - **Arquitectura de IA:** Patrón Adapter + Factory con Interfaces Puras TypeScript (**src/lib/ai/types.ts**).
- **Lógica de Servidor:** Server Actions para el procesamiento de reglas de negocio seguras.
- **Estilos:** TailwindCSS 4 para un diseño ligero, responsivo y de alta velocidad de renderizado.

2. ARQUITECTURA DE DATOS Y CONCURRENCIA

Para mitigar riesgos de sobreventa y colisiones de datos en un entorno de 4 terminales, la arquitectura traslada la lógica crítica al motor de la base de datos.

Lógica de Stock y Bloqueo Pesimista

El sistema implementa el Procedimiento Almacenado (RPC) `procesar_comanda()`. Este procedimiento ejecuta una transacción atómica que utiliza la cláusula `FOR UPDATE` (Bloqueo Pesimista). Al recibir un pedido, el sistema realiza una pasada preliminar sobre las filas de los productos o insumos origen involucrados bloqueándolas y validando la disponibilidad de stock *antes* de realizar la inserción de la cabecera en la tabla `Comandas`. De esta manera, si el stock es insuficiente, se ejecuta un Rollback inmediato sin alterar ni consumir números de la secuencia autoincremental (`numero_ticket`) del ticket de barra. Si todas las validaciones son correctas, el sistema descuenta las existencias, inserta el registro de cabecera consumiendo el número de ticket y graba los ítems de venta.

Lógica de Insumos y Stock Compartido (1:1)

Para optimizar el control de productos elaborados en el momento a partir de un recurso común y unitario (ej. pizzas de diversas variedades elaboradas a partir de una cantidad limitada de bollos de masa), el sistema soporta la asociación de stock compartido:

- **Enlace Relacional:** Un producto vendible (Pizza Muzza, Pizza Jamón) puede tener asociado un `insumo_compartido_id` apuntando al insumo origen (Bollo Pizza).
- **Deducción de Inventario:** Al venderse un producto enlazado, la deducción física del stock actual ocurre de forma automática y atómica sobre el insumo común en la base de datos.
- **Blindaje de Terminal contra Sobreventas:** El frontend realiza una acumulación en caliente de todos los ítems agregados al carrito de la terminal que compartan el mismo recurso. Si la sumatoria total del recurso iguala o supera su stock disponible, el sistema deshabilita las tarjetas de todas las variedades correspondientes e impide incrementar cantidades en el carrito local, bloqueando transacciones inválidas en caliente.

Stock Compartido vs. Productos de Elaboración Instantánea (Regla v1.9)

- **Stock Compartido (1 a 1):** Aplica exclusivamente cuando la relación insumo-producto es fija y previsible (ej. 1 bollo = 1 pizza de cualquier variedad).
- **Elaboración Instantánea (Venta Directa):** Productos finales con consumo variable de insumo (ej. Leche en Café, Hielo o Frutas en Coctelería) se configuran como vendibles sin seguimiento directo de stock (`controla_stock = false`). El insumo variable se controla como insumo interno y se audita manualmente durante el Cierre de Jornada.

Sincronización Realtime

Se utiliza Supabase Realtime (WebSockets) agregando las tablas `Comandas` y `Productos` a la publicación `supabase_realtime`. Cuando el `stockActual` cambia en la base de datos tras una venta o reposición, o se inserta una nueva comanda, todas las terminales reflejan el nuevo estado de los badges de stock y del listado de historial en tiempo real sin necesidad de refrescar el navegador.

3. LÓGICA DE SEGURIDAD REFORZADA, ROLES Y RLS

Jerarquía de Roles

- **Rol Admin:** Acceso total, incluyendo Auditoría, Historial, ABM de Productos, Cierre de Caja y Bot Asistente de IA.
- **Rol Empleado:** Acceso limitado exclusivamente a la toma de comandas y consulta de stock disponible.

JornadaGuard y Protección de Terminales (Bloqueo Realtime)

El sistema implementa el middleware `JornadaGuard` y un sistema de WebSockets para bloquear la navegación y funcionalidad de las terminales si no existe una jornada marcada como "Abierta". Esto evita registros accidentales fuera del horario operativo. Si el administrador cambia el estado de la jornada a `en_auditoria` o `cerrada` desde su terminal, la pantalla de los mozos es cubierta de inmediato por un banner opaco bloqueando la interfaz en tiempo real.

Flujo de Cierre y Seguridad Criptográfica

La transición de una jornada requiere la validación obligatoria del PIN de Administrador. En el servidor (`src/app/actions/jornada.ts`), la función `validarPinAdmin` aplica el algoritmo de hashing SHA-256 (`hashPin`) sobre la clave recibida antes de compararla contra la columna `pin` de la tabla `Usuarios`, garantizando la integridad criptográfica del acceso.

Seguridad a Nivel de Fila (RLS) Desglosada por Tabla

El sistema delega la autorización de acceso directamente en el motor de PostgreSQL para evitar adulteraciones de datos que esquiven el frontend:

- **Función Auxiliar `public.es_admin()`:** Se define con `SECURITY DEFINER` en PostgreSQL para consultar el rol `'Admin'` del usuario logueado en la tabla `Usuarios`, evitando bucles de recursión en RLS.
- **Bloqueo Administrativo Exclusivo:** Las tablas `Gastos` y `Auditoria_Inventario` requieren el rol `'Admin'` para cualquier operación (`SELECT`, `INSERT`, `UPDATE`, `DELETE`).
- **Integridad de Comandas:** Las tablas `Comandas` y `Comanda_Items` permiten `INSERT` a usuarios autenticados validando `auth.uid() = usuario_id`, pero restringen `UPDATE` y `DELETE` exclusivamente a administradores.
- **Catálogos y Menú:** `Productos`, `Categorias_Productos`, `Categorias_Gastos` y `Jornadas` admiten lectura libre (`SELECT`) para usuarios autenticados, pero restringen modificaciones a administradores.
- **Conocimiento Vectorial (`knowledge_chunks`):** Admite lectura y consulta para la ejecución de RAG por parte de usuarios autenticados y roles de servicio.

4. DICCIONARIO DE DATOS RELACIONAL Y VECTORIAL COMPLETO

A continuación se detalla la estructura relacional y vectorial de las 10 entidades de la base de datos de SAO Bar:

1. Entidad: Usuarios

- `id`: UUID (Primary Key). Mapeado directamente a la tabla interna de usuarios de Supabase Auth (`auth.users`) mediante FK.
- `email`: String (Único).
- `nombre`: String.
- `rol`: Enum ('Admin', 'Empleado').
- `pin`: Hash SHA-256 de 64 caracteres Hexadecimales.

- `activo`: Boolean. Default `true`.

2. Entidad: Categorías_Productos

- `id`: UUID (Primary Key).
- `nombre`: String (Único).
- `color_fondo`: String (Hexadecimal `#RRGGBB` validado con regex).
- `color_texto`: String (Hexadecimal `#RRGGBB` validado con regex).
- `activo`: Boolean. Default `true`.

3. Entidad: Categorías_Gastos

- `id`: UUID (Primary Key).
- `nombre`: String (Único, ej. *Insumos, Limpieza, Servicios, Otros, Personal, Mantenimiento*).
- `activo`: Boolean. Default `true`.

4. Entidad: Productos

- `id`: UUID (Primary Key).
- `nombre`: String.
- `categoria_id`: Foreign Key (`Categorías_Productos`) ON DELETE RESTRICT.
- `precio`: Numeric(10,2).
- `stockIdeal`: Integer (Referencia visual semanal).
- `stockInicial`: Integer (Actualizado en apertura/reposición).
- `stockActual`: Integer. Descontado por `procesar_comanda()` si `controla_stock = true`.
- `unidad`: String ('u', 'lt', 'ml').
- `activo`: Boolean. Default `true` (Baja lógica).
- `vendible`: Boolean (Determina si se muestra en el menú de comandas de mozos).
- `controla_stock`: Boolean (Determina si realiza seguimiento dinámico de inventario).
- `insumo_compartido_id`: UUID (Opcional). FK autorreferencial (`Productos`) ON DELETE SET NULL.

5. Entidad: Jornadas

- `jornada_id`: UUID (Primary Key).
- `estado`: Enum ('abierta', 'en_auditoria', 'cerrada').
- *Constraint de Unicidad Parcial*: `CREATE UNIQUE INDEX unique_active_jornada ON Jornadas (estado) WHERE estado IN ('abierta', 'en_auditoria');`.
- `fecha_inicio`: Timestamp con zona horaria.
- `fecha_fin`: Timestamp con zona horaria.
- `total_efectivo`: Numeric(10,2).
- `total_mp_lista`: Numeric(10,2).
- `total_mp_real`: Numeric(10,2).
- `comision_mp`: Numeric(10,2) (`total_mp_lista - total_mp_real`).
- `total_general`: Numeric(10,2) (`total_efectivo + total_mp_real`).
- `gastos_totales`: Numeric(10,2).

- `ganancia_neta`: Numeric(10,2) (`total_general - gastos_totales`).

6. Entidad: Comandas

- `comanda_id`: UUID (Primary Key).
- `jornada_id`: Foreign Key (`Jornadas`).
- `usuario_id`: Foreign Key (`Usuarios`).
- `numero_ticket`: SERIAL (Incremental autogestionado).
- `nro_beeper`: Integer. Opcional (Rango 1 a 20 inclusive).
- `fecha`: Timestamp con zona horaria.
- `total`: Numeric(10,2).
- `medio_pago`: Enum ('Efectivo', 'Mercado Pago', 'Regalo').

7. Entidad: Comanda_Items (N:N Break)

- `id`: UUID (Primary Key).
- `comanda_id`: Foreign Key (`Comandas`) ON DELETE CASCADE.
- `producto_id`: Foreign Key (`Productos`).
- `cantidad`: Integer.
- `precio_unitario_historico`: Numeric(10,2).

8. Entidad: Gastos

- `id`: UUID (Primary Key).
- `jornada_id`: Foreign Key (`Jornadas`).
- `categoria_id`: Foreign Key (`Categorías_Gastos`).
- `descripcion`: Text.
- `monto`: Numeric(10,2).

9. Entidad: Auditoria_Inventario

- `id`: UUID (Primary Key).
- `jornada_id`: Foreign Key (`Jornadas`).
- `producto_id`: Foreign Key (`Productos`).
- `conteo_fisico`: Integer.
- `unidades_utilizadas`: Integer (Ventidas o insumos consumidos).
- `unidades_regaladas`: Integer (Regalos de la casa).
- `stock_inicial`: Integer (Foto del stock de apertura).

10. Entidad: knowledge_chunks (v1.9 Infraestructura Vectorial)

- `id`: UUID (Primary Key).
- `titulo`: Text (Encabezado del fragmento procedimental).
- `contenido`: Text (Cuerpo de texto del conocimiento de negocio).
- `embedding`: Vector(768) NOT NULL (Incrustación de 768 dimensiones).
- `categoria`: Text (Default 'procedimiento').
- `created_at`: Timestamp con zona horaria.

- *Índice HNSW*: `CREATE INDEX ON knowledge_chunks USING hnsw (embedding vector_cosine_ops);`.
- *RPC SQL de Búsqueda Semántica*: `match_knowledge(query_embedding, match_threshold, match_count)`.

5. MÓDULO BOT ASISTENTE DE IA (V1.9)

Arquitectura RAG + Tool Use + Abstracción de Proveedores

El Asistente de IA combina dos fuentes de información en tiempo real:

1. RAG Procedimental (Base de Conocimiento Vectorizada):

- Consulta la tabla `knowledge_chunks` mediante la función RPC `match_knowledge` utilizando similitud coseno sobre vectores de 768 dimensiones.
- Proporciona respuestas precisas sobre reglas de negocio, apertura, cierre, auditoría y comandas.

2. Herramientas en Tiempo Real y Analítica Histórica (Tools SQL):

- `consultarStockActual({ productoNombre })`
- `consultarCajaJornada({ tipo })`
- `consultarGastosJornada({ tipo })`
- `consultarVentasJornada({ tipo })`
- `consultarCategorias()`
- `consultarCatalogoProductos({ categoriaNombre })`
- `consultarHistorialJornadas({ limite })`
- `consultarRendimientoHistoricoProducto({ productoNombre, limiteJornadas })`

Capa de Abstracción Multi-Proveedor (Adapter + Factory Pattern + Failover)

Para garantizar la independencia tecnológica (Vendor Lock-in zero) y la continuidad operativa a futuro:

- ``src/lib/ai/types.ts``: Define los contratos `IEmbeddingProvider` e `ILLMProvider`.
- ``src/lib/ai/embeddings.ts``: Implementa la fábrica `obtenerProveedorEmbeddings()`, soportando OpenRouter (`openai/text-embedding-3-small` / `nomic-embed-text`), Hugging Face Serverless (`BAAI/bge-base-en-v1.5`) y OpenAI directo.
- ``src/lib/ai/llm.ts``: Implementa la fábrica `obtenerProveedorLLM()`, soportando OpenRouter (`meta-llama/llama-3.3-70b-instruct`) como motor primario principal y Groq Cloud Native (`llama-3.3-70b-versatile`) como respaldo.
- **Resiliencia y Failover Bidireccional**: Cada petición es apátrida (*Stateless*). El sistema siempre intenta primero el proveedor primario (`openrouter`). Si este falla o carece de saldo, conmuta en caliente a Groq. En la siguiente consulta, el sistema intentará nuevamente OpenRouter de forma automática sin quedar bloqueado en el motor de respaldo.
- **Homogeneización de Hiperparámetros**: Todos los adaptadores transmiten explícitamente `temperature: 0.2`, `top_p: 0.9` y `max_tokens: 1024`.

Interfaz de Usuario Widget (Solapa de Archivero Emergente)

- **Ubicación & Acceso:** Implementado en `src/components/admin/AsistenteWidgetAdmin.tsx` e integrado en `AdminLayout.tsx`. Disponible exclusivamente para el rol Administrador al pie de pantalla.
- **Estado Plegado:** Solapa industrial mate (`#1A1A1A`) con texto **ASISTENTE**, indicador cian (`#30CFF2`) y flecha de despliegue (▲).
- **Estado Desplegado:** Tarjeta de chat (`w-[410px] h-[520px]`) con historial con scroll automático, chips de preguntas rápidas sugeridas e indicador de procesamiento.

Directivas de Seguridad y Guardrails del Asistente

El sistema aplica un blindaje de seguridad en el Prompt de Sistema mediante 8 directivas strictly enforced:

1. **Idioma Exclusivo:** Interacción y respuestas en idioma español únicamente.
2. **Delimitación de Dominio (Out-of-Domain):** Respuestas circunscritas exclusivamente a la operación, inventario, ventas y caja de SAO Bar 2026. Consultas ajenas son rechazadas con cordialidad.
3. **Prohibición Estricta de Alucinación (Zero-Hallucination):** Prohibido inventar datos, precios o existencias no presentes en RAG o herramientas SQL.
4. **Tono Cordial y Profesional:** Trato amable, educado y atento con la administradora.
5. **Formato Claro y Sin Emojis:** Respuestas estructuradas en español argentino sin el uso de emojis.
6. **Prudencia y Análisis:** Razonamiento reflexivo de la información antes de responder.
7. **Solo Lectura Estricta (Read-Only):** Prohibición absoluta de modificar, alterar o borrar registros.
8. **Prioridad Procedimental (RAG) vs Datos en Vivo (Tools):** Ante consultas sobre procedimientos o guías paso a paso (ej. ¿cómo elimino/desactivo un producto?, ¿cómo se hace el cierre?, ¿cómo funciona la calculadora?), se obliga al LLM a priorizar la explicación paso a paso del RAG. Si ejecuta una herramienta SQL, la respuesta final debe integrar obligatoriamente la explicación procedimental.

6. MÓDULO DE REPORTES Y ANALÍTICA DE JORNADAS

Generación Dinámica y Auditoría Anti-Robo

Los reportes se generan mediante JOINS relacionales en tiempo real. El motor de auditoría aplica la siguiente fórmula de conciliación de stock para productos con `controla_stock = true`:

- **Stock Teórico = Stock Inicial - Unidades Vendidas - Unidades Regaladas** (Leídos directamente de `Auditoria_Inventario`).
- **Desvío = Conteo Físico - Stock Teórico.**

Visualización de Desvíos y Ordenamiento

- **Token Error:** Si el Conteo Físico difiere del Stock Teórico, la interfaz resalta el registro con el color funcional Error (`#E2484A`).
- **Jerarquía de Datos:** Los reportes de stock se ordenan descendientemente por volumen de ventas.

Conciliación de Mercado Pago

Se realiza el cotejo financiero de Mercado Pago en el cierre:

- `comision_mp = total_mp_lista - total_mp_real.`
- **Token Advertencia:** Si `comision_mp` supera el 10% del `total_mp_lista`, se resalta en color Advertencia (`#BA7517`).

Módulo de Rendimiento y Rotación de Ventas

- **Consolidación en Memoria:** Agrupa relacionalmente en tiempo real todas las comandas de la jornada por `producto_id`.
- **Exclusión de Insumos y Regalos:** Filtra únicamente productos vendibles (`vendible = true`) e ignora comandas con medio de pago `'Regalo'` para evitar falsear las métricas de ingresos comerciales.

7. SISTEMA DE TOKENS VISUALES (SIN TABLAS)

Tipografía (Google Fonts)

- **Creepster:** Estilo Display para Logotipo "SAO Bar".
- **Livvic:** Estilo UI para toda la interfaz (botones, tablas, inputs, lecturas, reportes, historial, chat de IA).

Colores de Marca

- **Negro:** `#080A0D` (Fondo primario).
- **Naranja:** `#F26A1B` (Acciones primarias y paneles).
- **Naranja Alternativo:** `#F25922` (Interacciones y estados activos).
- **Cyan:** `#30CFF2` (Énfasis de navegación y logo).
- **Blanco:** `#F2F2F2` (Texto principal y contrastes).

Colores Funcionales

- **Efectivo:** `#1D9E75` (Confirmación y éxito).
- **Mercado Pago:** `#378ADD` (Pagos digitales).
- **Regalo / Cortesía:** `#7C3AED` (Regalos de la casa).
- **Error/Peligro:** `#E2484A` (Desvíos, alertas y errores).
- **Advertencia:** `#BA7517` (Alerta de stock bajo o comisiones fuera de lo normal).
- **Gris UI:** `#9D9D9D` (Textos secundarios y deshabilitados).

8. ESPECIFICACIONES DE INTERFAZ Y PANTALLAS

Layout General (Shell) y Menú Activo

- **Topnav (56px) / AdminNav:** Componente interactivo de cliente `<AdminNav />` para la navegación administrativa (Productos, Comandas, Caja, Cierre, Historial, Calculadora). Identifica la pestaña activa asignándole texto blanco `#F2F2F2` y borde inferior cian `#30CFF2`. Oculto en impresión (`print:hidden`).

Módulo ABM de Productos (Patrón Master-Detail)

- **Fondo y Tarjeta Central:** Fondo naranja corporativo (`bg-[#F26A1B]`) con menú administrativo en tarjeta oscura central (`bg-[#1A1A1A]`).
- **Grilla de Productos:** Muestra el catálogo categorizado. Productos inactivos se muestran al 40% de opacidad.
- **Barra de Filtros:** Búsqueda por texto, selector por categoría, selector de tipo de ítem y checkbox para listar inactivos.

- **Formulario de Edición/Alta:** Toggle de clasificación (Venta/Insumo), checkbox de seguimiento de stock y asociación de stock compartido (pizza → bollo).
- **Gestión de Categorías:** Modal para altas, bajas y edición de colores hexadecimales.

Terminal de Comandas

- **Sidebar Izquierdo (80px):** Selector de categorías vertical activo.
- **Grilla de Productos:** Tarjetas con badges de stock dinámico en tiempo real.
- **Subdivisión por Categorías:** Al filtrar por "Todos los productos", la grilla separa visualmente las categorías con líneas divisoras coloreadas.
- **Botonera de Pago:** Botones para Efectivo, Mercado Pago y botón discreto "Regalo de la Casa" (`h-8`, fuente negra cursiva seminegrita).
- **Impresión de Ticket de Cocina:** Modal de confirmación con botón de impresión optimizado para ticketera térmica de 76mm en blanco y negro (con metadatos de fecha localizada, número de ticket gigante, beeper opcional y ítems prefijados por `x`).
- **Scroll Adaptativo por Altura:** Listener de `resize` que ajusta desbordes si el alto de pantalla es inferior a `780px` (zoom 150-175%).
- **Cierre de Sesión Seguro:** Modal «Salir» con fondo desenfocado (`backdrop-blur-sm`).

9. PROCEDIMIENTOS OPERATIVOS CRÍTICOS

Envío de Comanda (Transacción Atómica RPC)

1. Validación de jornada abierta.
2. Bloqueo pesimista de filas (`FOR UPDATE`) sobre productos o insumos compartidos.
3. Validación de stock disponible (`stockActual >= cantidad`) solo si `controla_stock = true`.
4. Inserción de cabecera en `Comandas` y detalle en `Comanda_Items`.
5. Descuento de stock en tiempo real.
6. Si `medio_pago = 'Regalo'`, el total de la cabecera se fija en `0.00`.
7. Retorno del `numero_ticket` correlativo generado por la base de datos.

Flujo de Cierre de Jornada (4 Pasos Atómicos)

1. **Validación de PIN:** Hash SHA-256 en servidor (`validarPinAdmin`).
2. **Transición de Estado:** Cambio a 'En Auditoría' con bloqueo en tiempo real de terminales mozo.
3. **Carga de Gastos + Mercado Pago Real:** Registro de egresos y monto del posnet.
4. **Auditoría Física & Consolidación:** Carga de conteos reales en `Auditoria_Inventario`, cálculo de desvíos y actualización atómica del stock inicial y actual en `Productos` para la siguiente jornada.

10. ESPECIFICACIÓN TÉCNICA: MÓDULO CALCULADORA DE COSTOS (STATELESS)

Resumen Funcional & Flujo en Pantalla

- Simulador interactivo en cliente (React State) para presupuestar compras de insumos sin impactar la base de datos.
- Grilla interactiva con agregar/eliminar filas y autocompletado en caliente del catálogo real de productos.
- Aviso visual en cursiva simple: «*Aviso: Esta calculadora es una herramienta de simulación.*».

Estilos de Impresión A4

- Directivas `@media print` transforman la interfaz oscura en un reporte blanco A4.
- Se oculta la navegación, botones interactivos y bordes de inputs.
- Incorpora membrete institucional con logo circular `/images/sao-logo.png`, fecha localizada y pie de página en cursiva: «*Este documento es una simulación generada por la Calculadora de Costos de SAO Bar.*».

11. CASOS DE USO DETALLADOS

Caso de Uso: Tomar Comanda (CDU-SAO-001)

- **Actor/es:** Empleado (Mozo).
- **Precondiciones:** Jornada en estado 'abierta'. Productos seleccionados con `controla_stock = true` tienen `stockActual >= cantidad`. Beeper ingresado está en el rango de 1 a 20.
- **Flujo principal:**
 1. El mozo selecciona artículos del menú clasificados por categoría.
 2. El sistema actualiza optimistamente el stock en caliente deshabilitando temporalmente el botón si las unidades caen a 0.
 3. El mozo puede opcionalmente ingresar el número de beeper (1 al 20) y seleccionar el medio de pago, donde se incluye la opción de Regalo.
 4. Envía la comanda.
 5. Si el medio de pago es Regalo, el sistema notifica en pantalla la aclaración de cortesía y, al enviar la comanda, el procedimiento atómico `procesar_comanda` fuerza el total a `0.00` en la cabecera del ticket, registrando de igual forma la salida física del inventario para la auditoría de desvíos.
 6. Al confirmarse el pedido de forma exitosa, el sistema habilita un modal que muestra el número de ticket autoincremental asignado por la base de datos y el botón de Impresión de Comanda.
 7. El mozo presiona imprimir y el navegador despliega la ventana de impresión local con un diseño adaptado para comanderas térmicas monocromáticas de cocina (sin importes, con cantidades antepuestas por la letra x, beeper visible y fecha formateada localmente en español).
- **Postcondiciones:** Se decrementa el stock en base de datos. Si la comanda es de regalo, se registra con valor nulo en caja comercial, pero con persistencia de cantidades físicas e historial para auditoría interna.

Caso de Uso: Apertura de Jornada (CDU-SAO-002)

- **Actor/es:** Administrador.
- **Precondiciones:** La jornada anterior se encuentra en estado 'cerrada'. No existen jornadas activas.
- **Flujo principal:**
 1. El administrador ingresa el PIN correspondiente.

2. Define el saldo inicial de caja en efectivo y Mercado Pago.
3. Presiona abrir jornada.
- **Postcondiciones:** Transición a estado 'abierta', permitiendo la toma de comandas en todas las terminales.

Caso de Uso: Gestionar Productos (CDU-SAO-003)

- **Actor/es:** Administrador.
- **Descripción:** Alta, modificación y baja lógica de productos del menú o insumos.
- **Flujo principal:**
 1. El administrador accede a `/admin/productos`.
 2. Define nombre, precio, categoría, tipo (Venta/Insumo) y seguimiento de stock.
 3. Al editar, se puede actualizar el stock por reabastecimiento directo.
 - **Postcondiciones:** Actualización en base de datos. Si se da de baja un producto, pasa a `activo = false` ocultándose del menú pero preservando la integridad del historial.

Caso de Uso: Gestionar Categorías (CDU-SAO-004)

- **Actor/es:** Administrador.
- **Descripción:** Gestión de categorías de productos y sus colores de visualización.
- **Flujo principal:**
 1. El administrador abre el modal de categorías.
 2. Carga el nombre y los colores en formato Hexadecimal (validado por regex).
 - **Postcondiciones:** Si una categoría es desactivada, los productos vendibles pertenecientes a ella quedan bloqueados para su venta de forma automática.

Caso de Uso: Cierre de Jornada y Auditoría (CDU-SAO-005)

- **Actor/es:** Administrador.
- **Precondiciones:** Jornada en estado 'abierta'.
- **Flujo principal:**
 1. Ingreso de PIN de Administrador (Validado mediante hash SHA-256).
 2. Bloqueo transaccional de comandas (Jornada pasa a 'en_auditoria').
 3. Registro de egresos y el ingreso manual de posnet de Mercado Pago.
 4. En la sección de Auditoría Física de Inventario, el sistema clasifica de forma inteligente los productos. Si detecta un insumo con productos dependientes asociados (stock compartido), bloquea su edición de consumo manual y autocalcula su gasto sumando el consumo real de los productos finales vendidos.
 5. El sistema recupera el total de unidades regaladas durante la jornada y las deduce del stock teórico en conjunto con las ventas antes de calcular el desvío físico definitivo.
 - **Postcondiciones:** El sistema consolida la ganancia neta (efectivo + MP real - gastos), guarda la foto del stock de apertura y actualiza el stock actual de catálogo con los conteos físicos para servir de base para la próxima jornada.

Caso de Uso: Simulación de Costos (CDU-SAO-006)

- **Actor/es:** Administrador.
- **Descripción:** Simulación de compra o consumo en caliente de insumos y exportación en formato A4.
- **Flujo principal:**

1. Ingresa a la calculadora.
2. Carga filas de insumos con autocompletado y define cantidades e importes unitarios.
3. Compara totales recalculados.
4. Presiona Guardar / Imprimir.
- **Postcondiciones:** Se lanza el cuadro de diálogo de impresión del navegador renderizando el PDF A4 limpio en blanco, sin persistencia en base de datos.

Caso de Uso: Consulta al Bot Asistente de IA (CDU-SAO-007, v1.9)

- **Actor/es:** Administrador.
- **Precondiciones:** Usuario autenticado como Administrador.
- **Flujo principal:**
 1. El administrador ingresa a `/admin/asistente` o activa la solapa emergente `<AsistenteWidgetAdmin />`.
 2. Escribe una pregunta en lenguaje natural (ej. *"¿Cuánto vendimos hoy y cómo viene el stock de Fernet?"*).
 3. El sistema genera el embedding de la consulta y ejecuta `match_knowledge` (búsqueda semántica HNSW de 768 dimensiones) para recuperar el contexto procedimental relevante.
 4. Si la consulta requiere datos en vivo, el LLM invoca las herramientas SQL correspondientes (ej. `consultarStockActual`).
 5. El motor de IA ensambla el prompt aumentado y genera la respuesta respetando las 8 directivas de seguridad (Guardrails).
 6. El bot responde de forma profesional y concisa, sin emojis, integrando la explicación procedimental del RAG cuando corresponde.
- **Postcondiciones:** Respuesta visualizada en el chat con datos en tiempo real, sin alteración de registros (modo Read-Only).

Caso de Uso: Consultar Historial de Jornadas y Rendimiento (CDU-SAO-008)

- **Actor/es:** Administrador.
- **Precondiciones:** El usuario está autenticado con rol de Administrador. Existen jornadas en estado 'cerrada' registradas en el sistema.
- **Flujo principal:**
 1. El Administrador accede a la sección de Historial del panel de administración (`/admin/historial`).
 2. El sistema carga en memoria y renderiza el listado de jornadas cerradas de forma cronológica descendente, habilitando un filtro de fechas (Desde / Hasta) para acotar la búsqueda.
 3. El Administrador selecciona una jornada específica del panel lateral izquierdo.
 4. El sistema realiza la consulta a la base de datos y presenta la información de la jornada seleccionada distribuida en pestañas dinámicas: Resumen Financiero (flujo de caja completo, deducción automática de comisiones y balance de gastos), Auditoría de Inventario (Stock Inicial, unidades vendidas, regaladas, Stock Teórico, Conteo Físico y desvío resaltado en color de error si hay discrepancias), Gastos (descripciones, categorías y montos), Comandas (historial individual con ticket, hora, total y medio de pago) y Rendimiento (productos vendidos ordenados por rotación, excluyendo insumos operativos y comandas de regalo).
- **Postcondiciones:** El Administrador obtiene una vista detallada, inalterable y consolidada de los eventos financieros y de inventario de la jornada seleccionada.

Caso de Uso: Autenticación y Control de Acceso (CDU-SAO-009)

- **Actor/es:** Administrador / Empleado.
- **Precondiciones:** El usuario posee credenciales registradas en Supabase Auth y un perfil mapeado en la tabla **Usuarios**.
- **Flujo principal:**
 1. El usuario accede a la pantalla principal (/).
 2. Ingresa su correo electrónico y contraseña.
 3. El sistema valida las credenciales y recupera el perfil del usuario en la base de datos local.
 4. El sistema evalúa el estado del usuario: si está marcado como inactivo (**activo = false**), bloquea el acceso e interrumpe la sesión.
 5. Si el perfil es activo, el sistema redirige al usuario según su rol: Empleado → redirección forzada al área de toma de comandas (**/comandas**); Administrador → acceso completo habilitado a las rutas de gestión administrativa (**/admin/***).
- **Postcondiciones:** El usuario inicia sesión de forma segura y se aplican las restricciones de acceso a nivel de interfaz de usuario y políticas de base de datos (RLS).